Software Re-Engineering

Fall 2026

Lecture 4

Software Change

- Software Change Is Inevitable
- Software systems continuously evolve because of changing business and technical needs:
  - **New requirements emerge** as users interact with the software and their needs evolve.
  - **Business environments change**, including processes, regulations, markets, and organizational goals.
  - **Errors and defects must be identified and repaired** to maintain software quality.
  - **New hardware, platforms, and technologies** may need to be supported.
  - **Performance, security, usability, and reliability** may need to be improved.
  - **Integration requirements change** as organizations adopt new systems, services, and technologies.
- A key challenge for organizations is implementing and managing changes to legacy systems while maintaining their existing functionality, reliability, and business value.

Software Change Strategies

Software Maintenance

Architectural Transformation

Software Re-engineering

Software Maintenance

- Modify existing software to meet changing requirements while keeping its overall structure stable.
- Fix defects and bugs
- Adapt to new requirements or environments
- Improve performance or usability
- Add small enhancements without major architectural changes

Architectural Transformation

- Change the fundamental architecture of an existing software system.
- Restructure the system at the architectural level
- Example: Centralized → Distributed architecture
- May involve changes to components, communication, and deployment
- Used when the existing architecture no longer meets system needs

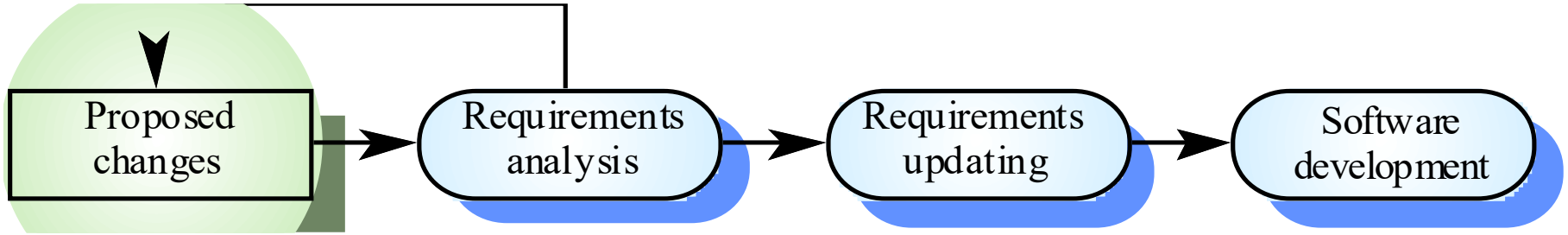
Software Re-engineering

- Restructure and modernize existing software to make it easier to maintain and evolve.
- Existing functionality is generally preserved
- Improve code structure, quality, and maintainability
- May include restructuring, reverse engineering, and migration
- Prepares the system for future changes

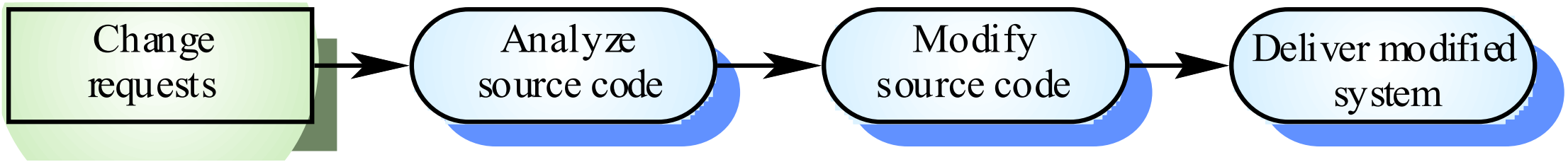
Change Requests

- A change request is a formal request to modify an existing software system. It may come from users, customers, management, developers, or other stakeholders.
- In principle, every change request should be reviewed, analyzed, prioritized, and approved before implementation to assess its impact, cost, risks, and urgency.
- However, some changes require immediate attention and may need to be implemented on an urgent basis:
  - **Fault Repair** – Fixing critical errors or defects that affect system operation.
  - **Environmental Changes** – Adapting the system to changes in hardware, operating systems, databases, platforms, or external systems.
  - **Urgent Business Changes** – Supporting critical changes in business processes, regulations, policies, or customer requirements.

Change Implementation



Emergency Change Implementation



Change Request (CR) Form

Capture | Evaluate | Implement

CR ID: _____

Date: _____

Field Name	
change_request_id	
title	A unique identifier of the CR
description	A concise summary of the CR
maintenance_type	A short description of the CR Classification of the maintenance type in terms of a member of {Corrective, Adaptive, Perfective, Preventive}

product_component	Product name
component	Component where the change is needed, or where the problem occurred

customer	
problem_origin	Name of the customer making the CR
impacts	The origin of the problem
resolution	Components that are affected by a change and its ripple effect
note	Documentation of what was changed, how, and why Additional information provided by the submitter for subsequent decision making

state

Present state of the CR in terms of a member of {Submit, Review, Analysis, Commit, Implementation, Verification, Closed, Declined}

software_release

The version number of the product release in which the CR is likely to be effective

committed_release

The version number of the product release in which the CR will be effective

priority

Priority of CR, which is an element of a set, namely, {normal, high}

severity

Severity of CR, which is an element of a set, namely, {normal, critical}

Software Maintenance

- Software maintenance is the process of modifying and supporting a software system after it has been delivered and put into operation.
  - It is mainly associated with custom software, while generic software products are typically evolved through new versions or releases.
  - Maintenance may involve fixing defects, adapting to environmental changes, improving performance, or implementing new requirements.
  - Most maintenance changes are made by modifying existing components or adding new components, rather than completely redesigning the system.
  - However, major architectural changes can also occur when a system needs significant modernization or restructuring.
  - Maintenance activities aim to preserve and improve the system's reliability, security, performance, and business value over its lifetime.
  - ISO/IEC 14764 defines software maintenance as the activities required to provide cost-effective support for a software system.
  - Maintenance-related activities can begin during the pre-delivery stage and continue throughout the post-delivery stage.

Why Modifications are Required



Market Conditions - Policies, which changes over the time, such as taxation and newly introduced constraints like, how to maintain bookkeeping, may trigger need for modification.



Client Requirements - Over the time, customer may ask for new features or functions in the software.

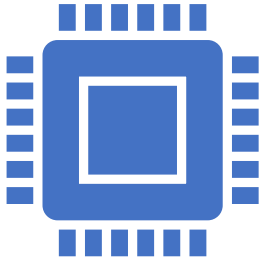


Host Modifications - If any of the hardware and/or platform (such as operating system) of the target host changes, software changes are needed to keep adaptability.



Organization Changes - If there is any business level change at client end, such as reduction of organization strength, acquiring another company, organization venturing into new business, need to modify in the original software may arise.

Maintenance Category

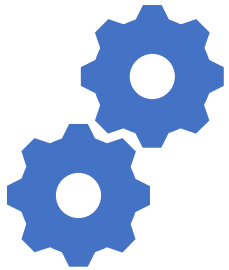


Maintenance to repair software faults

- Changing a system to correct deficiencies in the way meets its requirements

- Coding errors are usually relatively cheap to correct; design errors are more expensive as they may involve rewriting several program components.

- Requirements errors are the most expensive to repair because of the extensive system redesign which may be necessary.



Maintenance to adapt software to a different operating environment

- This type of maintenance is required when some aspect of the system's environment such as the hardware, the platform operating system, or other support software changes.

- The application system must be modified to adapt it to cope with these environmental changes.

Types of Maintenance

Corrective Maintenance

Adaptive Maintenance

Perfective Maintenance

Preventive Maintenance

Corrective Maintenance

- Corrective maintenance involves identifying and fixing defects, errors, or failures discovered after the software has been deployed.
- These problems may be reported by users, detected through testing, monitoring, or discovered by developers during system operation. The goal is to restore the software to its expected behavior.
- Examples:
  - Fixing an application that crashes when a user uploads a file.
  - Correcting an incorrect calculation in a billing system.
  - Fixing a login problem reported by users.

Adaptive Maintenance

- Adaptive maintenance involves modifying software so that it continues to work correctly when its environment changes.
- The environment may include the operating system, hardware, database, third-party services, APIs, programming platforms, business processes, or laws and regulations.
- Examples:
  - Updating an application to support a new operating system.
  - Modifying software when a third-party API changes.
  - Updating a system to comply with a new government regulation.
  - Adapting software to work with new hardware or database technology.

Perfective Maintenance

- Perfective maintenance involves making changes to improve, enhance, or extend an existing software system based on user needs or opportunities for improvement.
- Unlike corrective maintenance, the system may already be working correctly. The purpose is to make it more useful, efficient, user-friendly, or capable of meeting evolving requirements.
- Examples:
  - Adding a new reporting feature requested by users.
  - Improving the response time of a slow application.
  - Redesigning a difficult-to-use interface.
  - Adding search, filtering, or notification functionality.

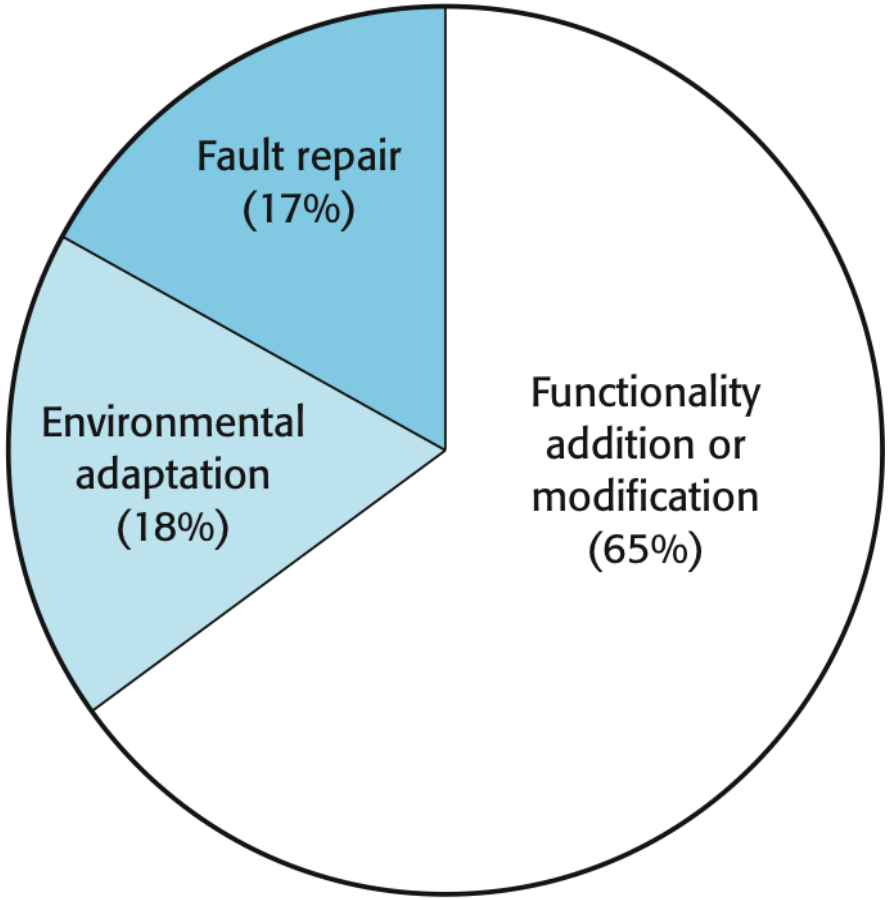
Preventive Maintenance

- Preventive maintenance involves making changes to reduce the possibility of future failures and make the software easier to maintain.
- These changes address weaknesses before they become serious problems. Developers may improve the internal structure of the software even when there is no immediate user-visible problem.
- Examples:
  - Refactoring complex or duplicated code.
  - Removing obsolete code and dependencies.
  - Improving documentation and code structure.
  - Updating vulnerable libraries before they cause security problems.
  - Improving automated tests to detect future defects earlier.

Types of Maintenance

- Corrective fixes today's problems, Adaptive responds to external changes, Perfective improves the system, and Preventive prepares the system for tomorrow.

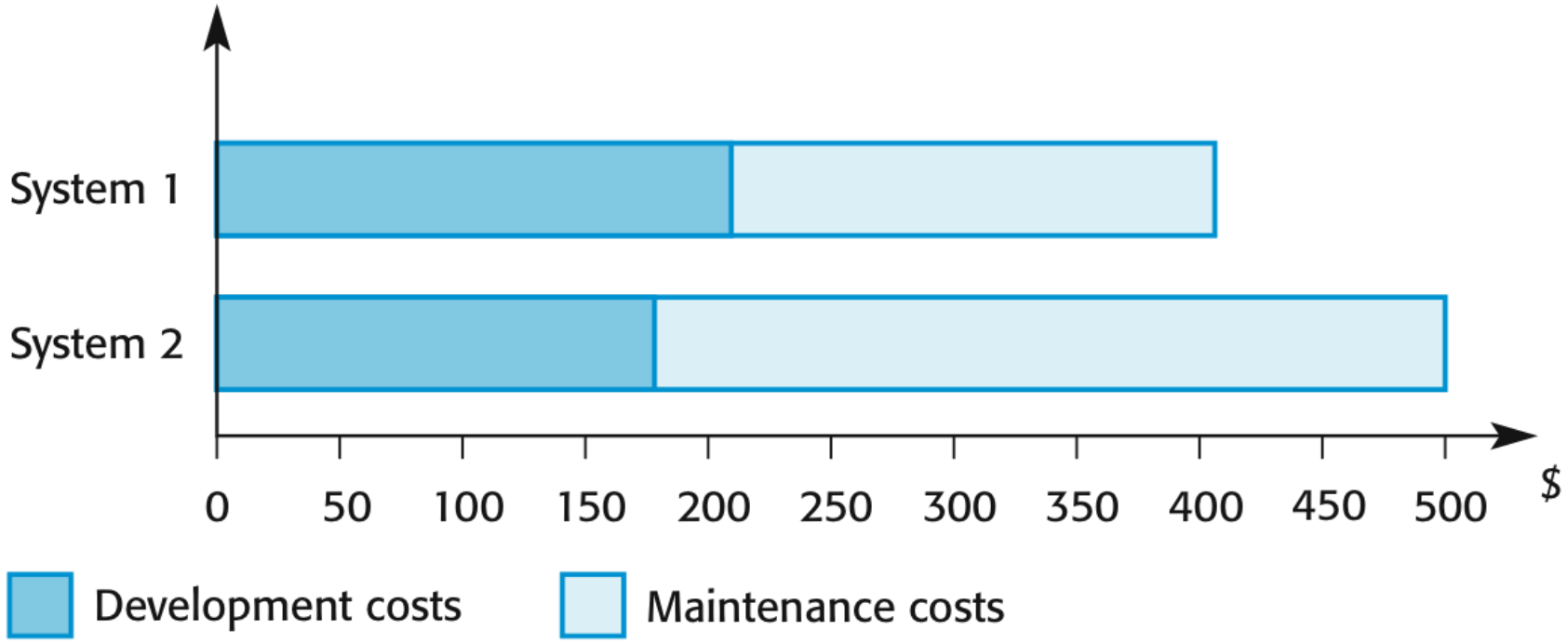
Maintenance Effort Distribution



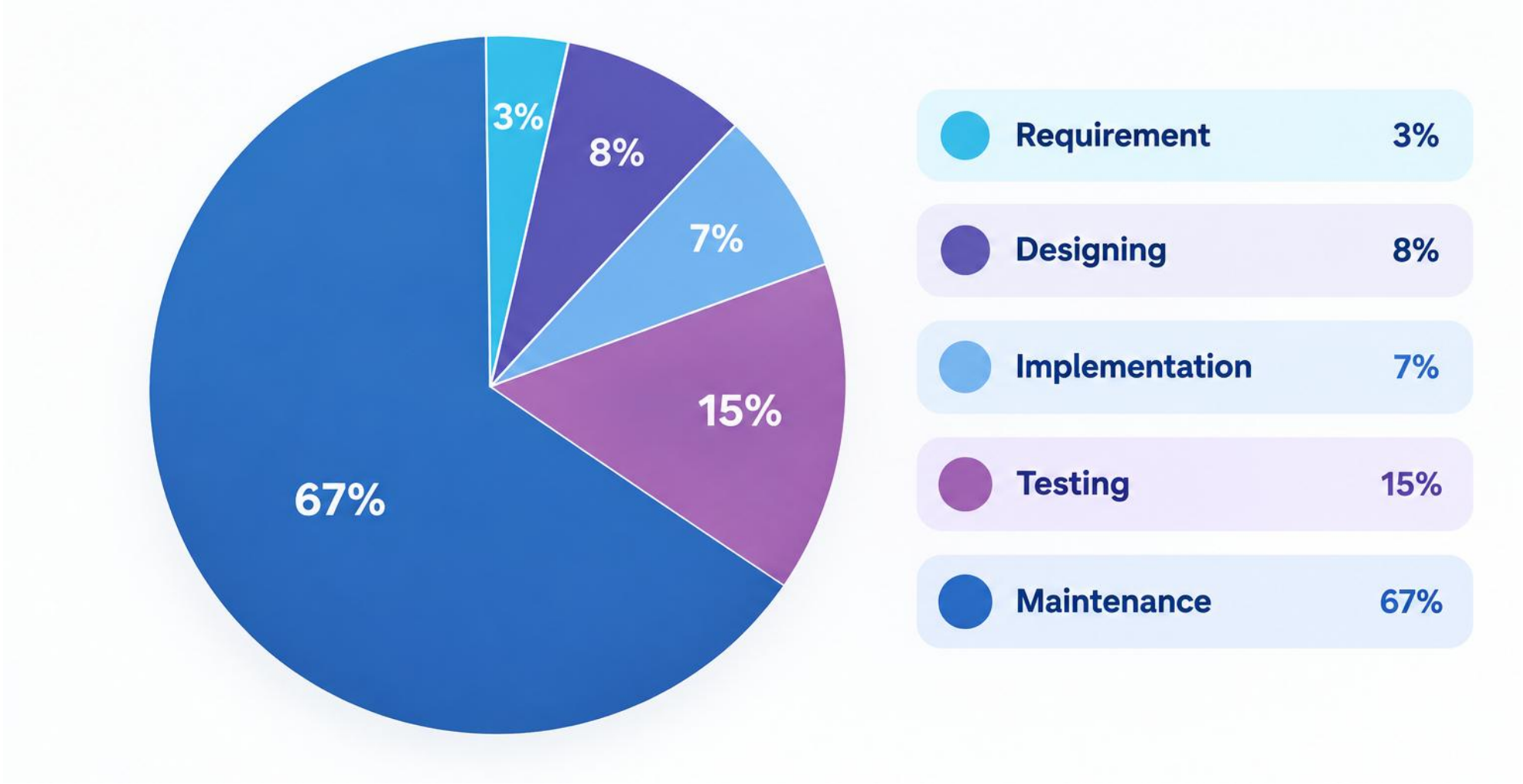
Software Maintenance Costs

- Software maintenance can account for a significant portion of the total cost of a software system throughout its lifetime. In many long-lived systems, cumulative maintenance and evolution costs can exceed the original development cost.
  - **Maintenance is a long-term activity:** A software system may be maintained for many years after its initial development.
  - **Costs accumulate over time:** New requirements, defect fixes, technology changes, testing, deployment, and support all contribute to the overall maintenance cost.
  - **Poorly managed changes increase complexity:** Frequent modifications without proper design, refactoring, documentation, and testing can gradually degrade the software structure, making future changes more difficult and expensive.
  - **Legacy systems can be expensive to support:** Older systems may depend on outdated programming languages, operating systems, databases, compilers, hardware, or specialized skills.
  - **Understanding the existing system takes time:** Developers may need to study unfamiliar or poorly documented code before making changes.
  - **Testing and regression risks increase:** Changes to one part of a large system may affect other components, requiring extensive testing.

Development and maintenance costs



Cost of Maintenance (AVG)



Maintenance Cost Factors

Team stability	• Maintenance costs are reduced if the same staff are involved with them for some time
Contractual responsibility	• The developers of a system may have no contractual responsibility for maintenance so there is no incentive to design for future change
Staff skills	• Maintenance staff are often inexperienced and have limited domain knowledge
Program age and structure	• As programs age, their structure is degraded and they become harder to understand and change

Real-World Factors Affecting Maintenance Cost

Age of the system	Outdated technology	Limited system knowledge
Increasing complexity	Poor documentation	Compatibility and integration



Questions?